

TEMPLATE PARA ENTREGA DO PROJETO DA DISCIPLINA

DevOPs na Prática

Fase 2: Entrega Contínua, Monitoramento e Segurança

NOME DO ESTUDANTE:

Peterson Weschenfelder do Amaral

SEÇÃO 1

Pipeline de Entrega Contínua

a) EXPANSÃO DO PIPELINE DE CI PARA INCLUSÃO DA ENTREGA CONTÍNUA (CD):

Evolução Arquitetural: De Integração para Entrega Contínua Segura

O pipeline evoluiu de uma única etapa de testes para uma esteira de 5 estágios automatizados e encadeados via GitHub Actions:

Integração Contínua (CI): Validação automática (testes unitários Node.js) a cada commit ou PR.

Build & Container Registry: Construção e publicação da imagem no GitHub Container Registry (GHCR), rigorosamente tagueada com o SHA do commit para rastreabilidade.

Security Scan (Shift-Left Security): Análise automática de vulnerabilidades via Trivy na imagem recém-construída, bloqueando deploys inseguros.

Deploy Staging: Implantação automática em ambiente de homologação via shell scripts com validação via Health Checks.

Deploy Produção: Entrega controlada via aprovação manual (GitHub Environments), garantindo governança antes de impactar o usuário.

SEÇÃO 2

Implementação de Containers e Orquestração

a) CONTAINERIZAÇÃO DA APLICAÇÃO UTILIZANDO DOCKER:

Insira o link de acesso ao repositório (GitHub).

Focado em Segurança e Performance

Arquitetura Multi-estágio (Multi-stage Build): Separação clara entre o ambiente de build e o ambiente de runtime (Node 20 Alpine), resultando em imagens ultraleves.

Hardening de Segurança: Execução da aplicação delegada a um usuário não-root (nodeuser), minimizando vetores de ataque em caso de invasão do container.

Resiliência (Self-Healing básico): Instrução HEALTHCHECK nativa no Dockerfile, monitorando a saúde da aplicação ativamente.

Paridade de Ambientes: Orquestração via docker-compose.yml utilizando *profiles* isolados (prod, dev) para garantir que o desenvolvimento reflita com precisão o ambiente de produção.

Link Dockerfile: <https://github.com/PetersonWA/devOps-Fase1/blob/main/Dockerfile>

Link Compose: <https://github.com/PetersonWA/devOps-Fase1/blob/main/docker-compose.yml>

b) SCRIPTS DE DEPLOY USANDO OS CONTAINERS:

Insira o link de acesso ao repositório (GitHub).

Automação e Resiliência na Camada de Implantação

Deploy Zero-Touch (deploy.sh):

- Pull da imagem exata validada no pipeline.
- Substituição do container (Stop -> Remove -> Run) injetando as variáveis de ambiente necessárias.
- Validação rigorosa: 5 tentativas automáticas de Health Check. O pipeline aborta se o container subir degradado.

Estratégia de Recuperação Rápida (rollback.sh):

- Acionado em caso de falha no deploy ou regressão em produção.
- Retorno imediato à última versão estável utilizando a variável dinâmica PREVIOUS_TAG.
- Health check obrigatório pós-rollback para garantir a estabilidade do serviço restaurado.

Link Deploy: <https://github.com/PetersonWA/devOps-Fase1/blob/main/scripts/deploy.sh>

Link Rollback: <https://github.com/PetersonWA/devOps-Fase1/blob/main/scripts/rollback.sh>

SEÇÃO 3

Relatório Final e Demonstração

a) RELATÓRIO DETALHADO DAS ETAPAS DO PROJETO → FASE 1

Fundação: Integração Contínua e Infraestrutura como Código (IaC)

Governança de Código: Repositório Git estabelecido com proteção na branch main.

Pipeline de CI: Configuração do GitHub Actions para testes automatizados de conexão e validação estrutural a cada push/PR.

Infraestrutura Declarativa: Provisionamento via Terraform (AWS EC2 t2.micro + S3 bucket), garantindo a criação de ambientes idênticos e reproduzíveis.

a) RELATÓRIO DETALHADO DAS ETAPAS DO PROJETO → FASE 2

Expansão: Entrega Contínua, Segurança e Governança

Esteira de CD: Expansão do workflow para 5 estágios completos (CI → Build → Scan → Staging → Produção).

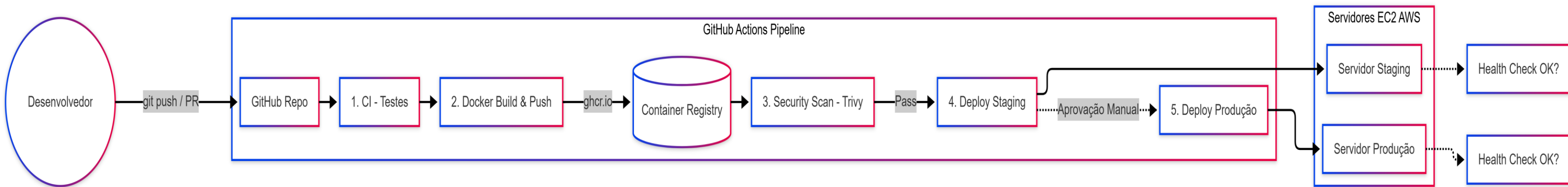
Containerização Eficiente: Envelopamento da aplicação via Docker com foco em segurança (non-root) e performance.

Segurança Integrada: Ferramenta Trivy embutida no pipeline para barrar imagens com vulnerabilidades críticas (CRITICAL/HIGH).

Governança de Deploy: Implementação de Portões de Aprovação (Manual Approvals) via GitHub Environments para o ambiente de Produção.

b) DEMONSTRAÇÃO PRÁTICA DE TODO O FLUXO DEVOPS IMPLEMENTADO:

Insira aqui a representação visual do Fluxograma completo com a demonstração de todas as etapas do pipeline de integração.



c) ANÁLISE DOS RESULTADOS E SUGESTÃO DE MELHORIAS FUTURAS:

Resultados Alcançados (Impacto Direto):

- **Automação End-to-End:** Redução drástica do *Lead Time for Changes*, indo do commit à produção sem intervenção manual não planejada.
- **Eficiência e Segurança:** Imagens finais otimizadas (~50MB) rodando com postura de segurança elevada.
- **Confiabilidade Máxima:** Deploys atrelados a hashes criptográficos (SHA) com capacidade de rollback aferido em segundos.

Roadmap Arquitetural (Próximos Passos Estratégicos):

- **Observabilidade Total:** Implantação de Prometheus + Grafana para métricas e stack ELK para logs centralizados.
- **Alta Disponibilidade (HA):** Evoluir a orquestração do Docker Compose para Kubernetes (K8s), permitindo escalabilidade horizontal automática.
- **Segurança de Credenciais (Secrets):** Migração de variáveis de ambiente locais para um cofre de senhas dedicado (ex: AWS Secrets Manager).
- **Estratégia Zero-Downtime:** Evoluir o script de deploy para um padrão de implantação Blue/Green ou Canary.

GRADUAÇÃO
PUCRS online